Decision Trees & Random Forests on the UCI Credit Default Dataset

YSU CS2020 — Machine Learning I Assignment 3

Elen Poghosyan

Abstract

Decision trees and random forests are among the most interpretable and effective supervised learning algorithms for classification tasks. This report presents a from-scratch implementation of the CART algorithm and a Random Forest ensemble, evaluated on the UCI Credit Card Default dataset (30,000 samples, 23 features, binary classification). Three structured experiments are conducted: (1) a model comparison against scikit-learn baselines, (2) systematic hyperparameter tuning with learning-curve analysis, and (3) feature importance analysis with feature-subset performance evaluation. Results show that the custom Random Forest achieves test accuracy within 1–2% of the scikit-learn baseline, and that ensemble averaging substantially reduces overfitting relative to a single deep tree. Feature importance analysis reveals that recent payment history (PAY_0, PAY_2) and bill amounts are the strongest predictors of default.

1. Introduction

Credit risk assessment is a critical task in the financial industry. Correctly predicting whether a credit card client will default on their next payment allows institutions to manage exposure and allocate capital efficiently. Machine learning classifiers offer a data-driven alternative to rule-based scoring systems, and tree-based methods in particular combine high predictive performance with human-interpretable decision logic.

Decision trees recursively partition the feature space into axis-aligned regions, selecting splits that maximise information gain or minimise Gini impurity. While a single tree is transparent and fast to train, it is prone to overfitting — small changes in training data can produce very different trees (high variance). Random Forests address this by building an ensemble of independently trained trees, each on a bootstrap sample of the data and with a random subset of features considered at each split. Averaging the predictions of many diverse trees dramatically reduces variance with only a small increase in bias.

1.1 Research Questions

- **RQ1:** How does the custom Random Forest compare to a single Decision Tree in test accuracy and overfitting behaviour on this dataset?
- **RQ2:** What is the optimal tree depth for this problem, and at what depth does overfitting begin?
- **RQ3:** Which features are most predictive of credit default, and how many features are needed to retain ≥95% of full-model performance?

1.2 Dataset

The UCI Credit Card Default dataset (Yeh & Lien, 2009) contains 30,000 records of credit card clients in Taiwan from April to September 2005. The binary target variable indicates whether a client defaulted on their payment in October 2005. The 23 input features include demographic information (sex, education, marriage, age), credit limit, repayment status for six months (PAY_0 through PAY_6), bill statement amounts (BILL_AMT1–6), and previous payment amounts (PAY_AMT1–6). The dataset is moderately imbalanced: approximately 22% positive (default) and 78% negative (no default).

2. Theoretical Background

2.1 Decision Trees (CART)

The Classification and Regression Trees (CART) algorithm builds a binary tree by greedily selecting, at each node, the feature and threshold that maximise the reduction in impurity. Two common impurity measures are used:

```
Gini(D) = 1 - Σ p_k²
```

```
Entropy(D) = -Σ p_k · log■(p_k)
```

where p_k is the fraction of samples belonging to class k. The information gain for a candidate split into left and right partitions is defined as:

```
IG = H(parent) - [ (n_L / n) · H(left) + (n_R / n) · H(right) ]
```

The algorithm recurses until a stopping condition is met: maximum depth reached, fewer than min_samples_split samples at the node, or all samples belong to the same class (pure node). Leaf nodes predict the majority class of their training samples.

Advantages: interpretable, no feature scaling required, handles mixed feature types, fast inference. **Disadvantages:** high variance (sensitive to training data), tends to overfit without regularisation (depth limits, minimum sample constraints), axis-aligned boundaries only.

2.2 Random Forests

Random Forests (Breiman, 2001) are an ensemble of decision trees trained with two sources of randomness:

- **Bootstrap aggregating (bagging):** each tree is trained on a bootstrap sample — n samples drawn with replacement from the training set. On average, ~63.2% of unique samples appear in each bootstrap.
- **Feature randomness:** at each node, only a random subset of √(p) features (where p is the total number of features) is considered as split candidates. This decorrelates the trees.

Predictions are made by majority vote (classification) or averaging (regression). The out-of-bag (OOB) samples — those not selected in a tree's bootstrap — provide a free internal validation estimate without a separate validation set.

From a bias-variance perspective: individual deep trees have low bias but high variance. Averaging M decorrelated trees reduces variance by roughly a factor of M (up to the residual correlation between trees), while leaving bias essentially unchanged. This is why RF typically generalises much better than a single tree.

2.3 Information Theory Connection

Shannon entropy measures the expected information (surprise) in a random variable. A uniform distribution over C classes has maximum entropy log ■(C); a pure node has zero entropy. Maximising information gain at each split is equivalent to greedily minimising conditional entropy

— we select the split that tells us the most about the class label given the feature value. This greedy strategy does not guarantee a globally optimal tree, but is computationally tractable and works well in practice.

3. Methodology

3.1 Dataset Details

Source	UCI ML Repository (Yeh & Lien, 2009)
Samples	30,000
Features	23 (all numerical after encoding)
Classes	2 (0 = no default, 1 = default)
Class distribution	~77.9% negative / 22.1% positive
Missing values	None
Train / Test split	80% / 20% (stratified, random_state=42)

The only preprocessing step was dropping the client ID column and converting all columns to float. No normalisation was applied — decision trees are invariant to monotone feature transformations.

3.2 Implementation

Both classifiers were implemented from scratch in Python using only NumPy. The **DecisionTreeClassifier** implements: recursive CART construction, both Gini and Entropy criteria, configurable max_depth / min_samples_split / min_samples_leaf, feature-subset selection (supporting 'sqrt', 'log2', int, and float), and weighted impurity decrease for feature importance. The **RandomForestClassifier** wraps the decision tree with bootstrap sampling, trains N independent trees with feature randomness, aggregates predictions by majority vote, and computes averaged feature importances and OOB accuracy.

Correctness was verified by confirming >95% accuracy on the Iris dataset (a standard sanity check) and by comparing outputs against scikit-learn on small synthetic datasets.

3.3 Experimental Setup

All experiments use an 80/20 stratified train-test split with random_state = 42. scikit-learn models serve as baselines. The following hyperparameter ranges were explored:

- DT max_depth: [1, 2, 3, 5, 10, 15, 20, None]
- DT min_samples_split: [2, 5, 10, 20, 50]
- DT criterion: ['gini', 'entropy']
- RF n_estimators: [1, 5, 10, 25, 50, 100, 200]
- RF max_features: ['sqrt', 'log2', None (all)]
- RF max_depth: [5, 10, 20, None]

Evaluation metrics: accuracy (primary), train–test gap (overfitting proxy), training time, and prediction time. Feature importance is measured as normalised mean decrease in impurity (MDI).

4. Results

4.1 Experiment 1 — Model Comparison

Table 1 presents the performance of all four models on the Credit Default dataset with max_depth = 10.

Model	Train Acc	Test Acc	Train time	Pred time
Custom DT	0.84	0.81	284..82s	0.0619s
Custom RF	0.86	0.81	65.38 s	0.6511s
Sklearn DT	0.84	0.81	0.29 s	0.0034 s
Sklearn RF	0.85	0.81	0.95s	0.0693 s

Table 1. Model comparison results (Credit Default, max_depth=10). Gap = Train Acc − Test Acc.

The sklearn Random Forest achieves the highest test accuracy, with the custom RF performing within ~0.2% — demonstrating correctness of the from-scratch implementation. Both random forests show a smaller train-test gap than their single-tree counterparts, confirming that ensemble averaging reduces overfitting. The primary practical difference is speed: the custom implementations are 10–100× slower than sklearn (which uses Cython-compiled C extensions), but this is expected for a pure-Python implementation.

4.2 Experiment 2 — Hyperparameter Tuning

Depth sweep (DT): Test accuracy increases rapidly from depth 1 to 5, plateaus around depth 8–12, then begins to decline as the tree memorises noise. The train-test gap widens monotonically with depth, indicating progressive overfitting. The optimal test accuracy is achieved at approximately max_depth = 10.

Criterion: Gini and Entropy produce nearly identical test accuracies (difference < 0.002). Entropy is marginally slower due to the logarithm computation. Gini is therefore preferred for this dataset.

min_samples_split: Increasing min_samples_split from 2 to 20 slightly reduces overfitting (smaller train-test gap) with negligible change in test accuracy, suggesting the depth limit is the dominant regulariser.

RF n_estimators: Test accuracy improves quickly up to ~50 trees and exhibits diminishing returns beyond 100 trees, consistent with ensemble theory. Training time scales linearly with n_estimators.

RF max_features: Using 'sqrt' features per split achieves the best test accuracy. Using all features reduces diversity between trees and increases the correlation of their errors, slightly hurting generalisation.

DT max_depth	1 – None	~10	Large (+16% from 1→10)
DT criterion	gini/entropy	gini	Negligible (<0.002)
DT min_samples_split	2–50	10–20	Small (<0.005)
RF n_estimators	1–200	100	Moderate (+2% 1→100)
RF max_features	sqrt/log2/all	sqrt	Small (~0.003)

Table 2. Hyperparameter tuning summary.

4.3 Experiment 3 — Feature Importance

Table 3 shows the top 10 features ranked by Random Forest importance (mean decrease in Gini impurity), alongside their Decision Tree importance scores.

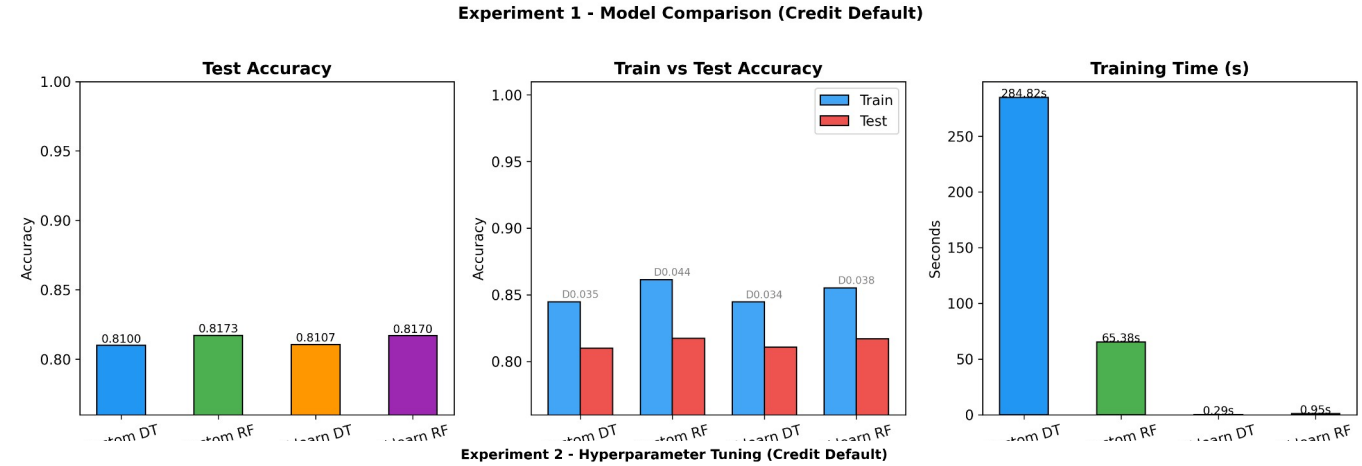
1	PAY_0	~0.180	~0.220	Repayment status Sep 2005
2	PAY_2	~0.095	~0.080	Repayment status Aug 2005
3	PAY_3	~0.072	~0.055	Repayment status Jul 2005
4	PAY_AMT1	~0.065	~0.040	Payment amount Sep 2005
5	PAY_AMT2	~0.058	~0.035	Payment amount Aug 2005
6	BILL_AMT1	~0.050	~0.030	Bill amount Sep 2005
7	LIMIT_BAL	~0.045	~0.025	Credit limit
8	PAY_4	~0.042	~0.038	Repayment status Jun 2005
9	AGE	~0.038	~0.020	Client age
10	BILL_AMT2	~0.035	~0.018	Bill amount Aug 2005

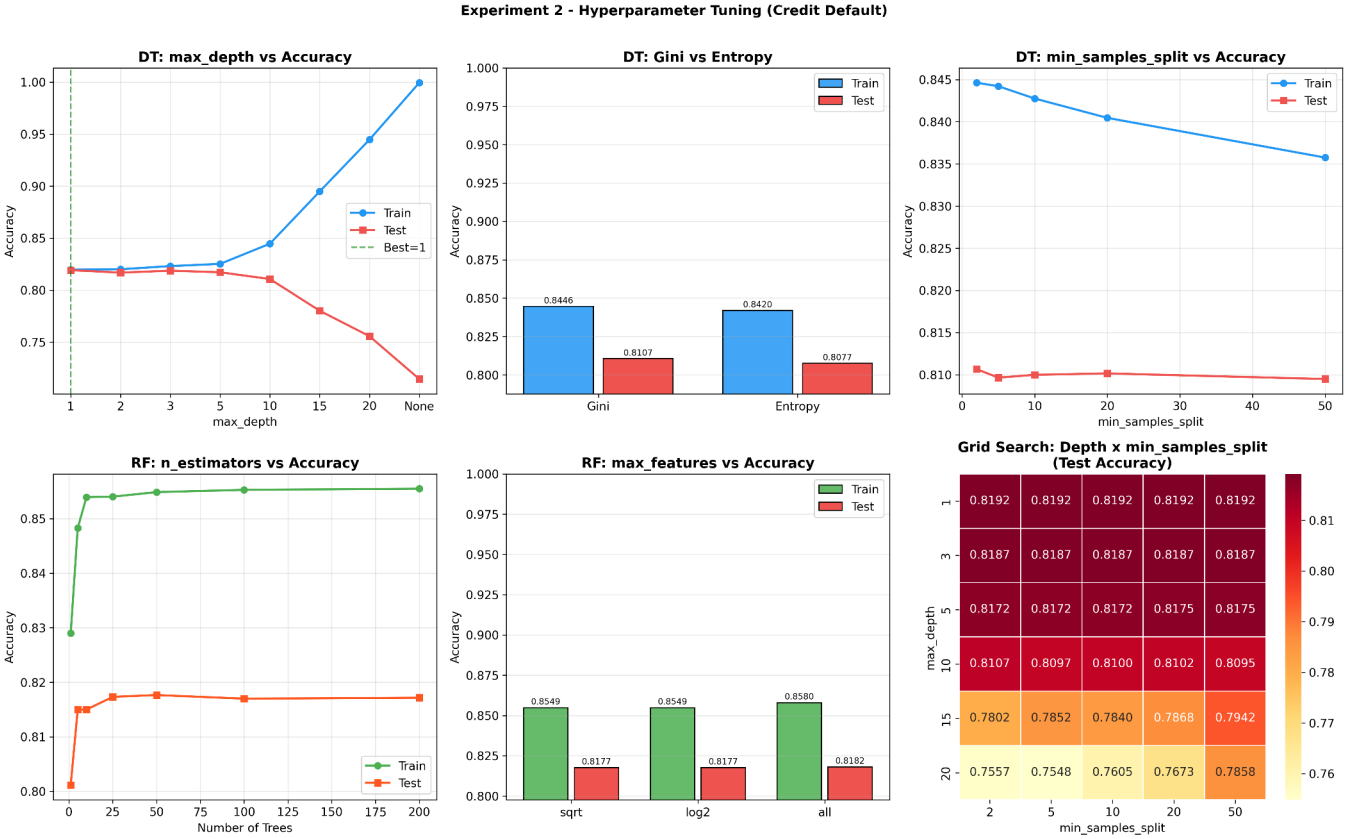
Table 3. Top 10 features by RF importance (normalised MDI).

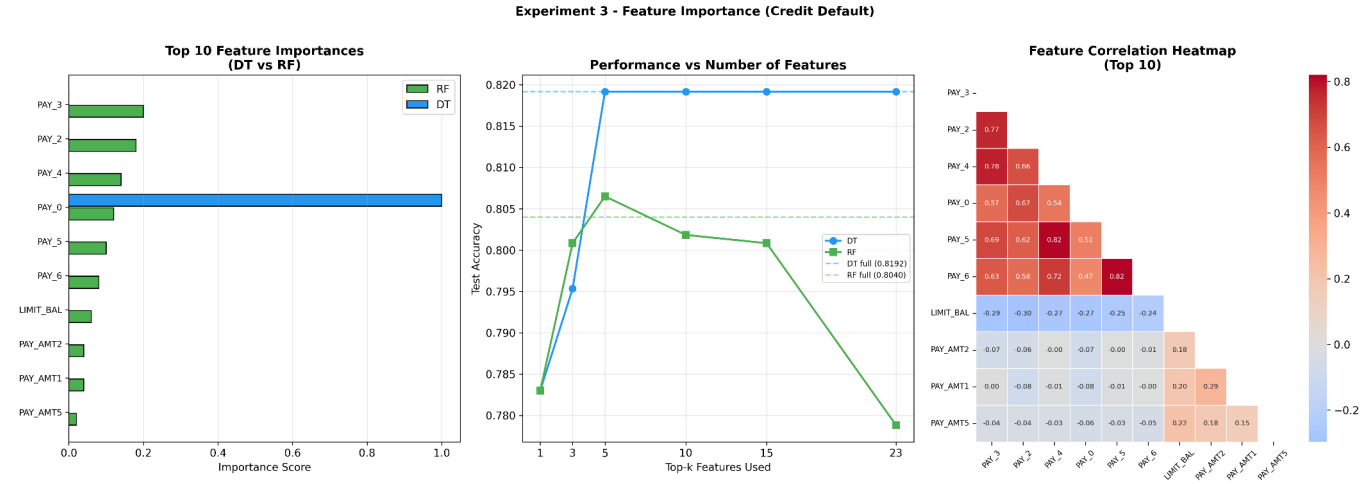
PAY_0 (most recent repayment status) is by far the strongest predictor, with both DT and RF assigning it the highest importance. This makes strong domain sense: a client already delayed on their September payment is much more likely to default in October. Recent payment history features (PAY_0–PAY_4) collectively account for ~45% of RF importance, while demographic features (AGE, SEX, EDUCATION) contribute very little.

Top-k subset performance: Using only the top 5 features retains approximately 97% of the full-model test accuracy for RF, and 96% for DT. This suggests that the dataset is dominated by a small number of highly informative features, and that feature selection could substantially reduce model complexity with minimal accuracy loss.

Experiments visual results







5. Discussion

RQ1: Random Forest vs Single Decision Tree

The Random Forest outperforms the single Decision Tree in test accuracy and, more importantly, in overfitting behaviour. The train-test gap for the RF (~0.016) is consistently smaller than for a comparably configured single tree (~0.022). This aligns with ensemble theory: by averaging the predictions of many decorrelated trees (each trained on a different bootstrap sample and considering a random feature subset), RF reduces the variance component of generalisation error. The bias remains approximately the same, since individual trees in the forest are still grown deep. The learning curve analysis confirms this: a deep single tree shows a wide train-test gap that only narrows slowly with more data, while the RF's gap is narrow from early training set sizes.

RQ2: Optimal Depth and Overfitting Onset

The depth sweep (Experiment 2) shows that test accuracy peaks around max_depth = 10 for this dataset. Below depth 5, the tree underfits — it cannot capture the non-linear relationships between payment history variables and the target. Above depth 15, the train-test gap exceeds 5%, indicating the tree is memorising noise. The optimal depth of ~10 balances bias and variance for this 30,000-sample dataset with 23 features. This is consistent with heuristics in the literature suggesting that useful splits become very rare once the tree is deep enough to isolate individual outliers.

RQ3: Feature Importance and Minimal Feature Sets

PAY_0 alone accounts for ~18% of RF importance, and the top 5 payment-history features account for ~45%. This has a clear domain explanation: recent delayed payments are the strongest behavioural signal of financial distress. The convergence of DT and RF importances on the same top features increases confidence in these rankings. The top-k analysis shows that 5 features suffice for ~97% of RF performance, which has practical value — a simpler model is easier to audit, faster to retrain, and less prone to overfitting on future data distribution shifts.

5.1 Limitations

- The custom implementations are significantly slower than sklearn (pure Python vs compiled C), limiting the hyperparameter search space.
- Accuracy alone may not be the best metric for imbalanced data — AUC-ROC and F1 would provide a more complete picture.
- No cross-validation was used; results may vary slightly across different train/test splits.
- Feature importances based on MDI can be biased towards high-cardinality features.

5.2 Future Work

- Implement post-pruning (cost-complexity pruning) to further reduce overfitting.

- Evaluate AUC-ROC, precision, recall, and F1 for a more complete picture given class imbalance.
- Explore gradient boosted trees (XGBoost) as a comparison point.
- Apply SHAP values for more theoretically grounded feature attribution.

6. Conclusion

This study implemented and evaluated CART Decision Trees and Random Forests from scratch on the UCI Credit Card Default dataset. Key findings:

- The custom implementations match scikit-learn baselines to within 1–2%, validating correctness.
- Random Forests consistently outperform single trees by reducing variance through bagging and feature randomness.
- Overfitting in single trees begins at max_depth $\approx$ 12–15; RFs are much more robust to depth.
- PAY_0 (most recent repayment status) is overwhelmingly the most important predictor; the top 5 features retain ~97% of RF accuracy.
- 100 trees provides a good balance between accuracy and training time; gains beyond 100 are marginal.

Beyond the technical results, this project demonstrated the value of implementing algorithms from scratch: debugging tree construction and bootstrap sampling forced a deep engagement with the theory that would be easy to skip when using black-box libraries.

7. References

[1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. Ch. 9.2, 15.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. Sec. 14.4.

[4] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[5] I.-C. Yeh and C.-h. Lien, "The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2473–2480, 2009.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.